

ASSOCIATION ANALYSIS

FREQUENT ITEMSET GENERATION AND RULE GENERATION – PART I

Fabio Stella

Associate Professor

c/o Department of Informatics, Systems and Communication

University of Milano-Bicocca



FREQUENT ITEMSET GENERATION

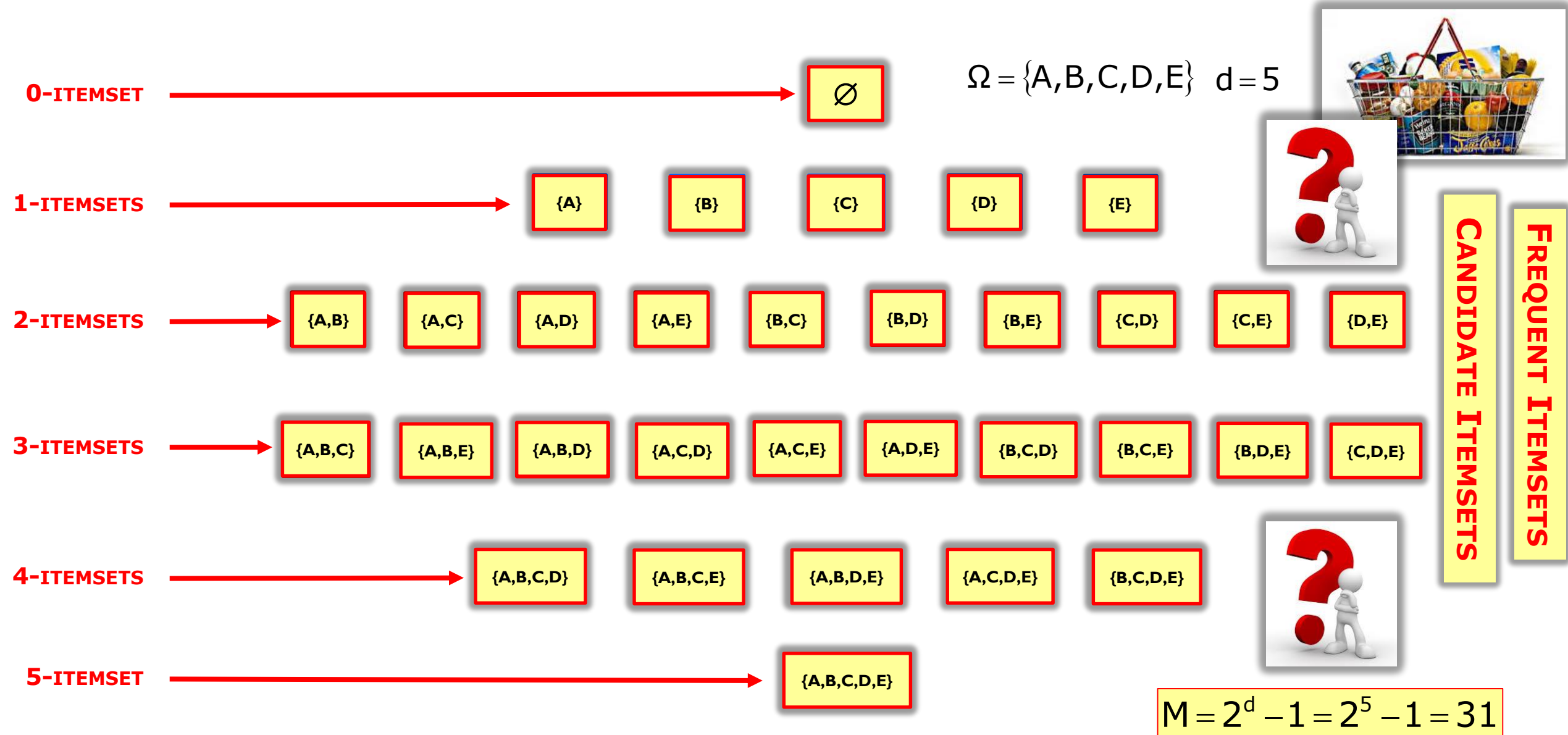
The following concepts will be introduced:

- ✓ **CANDIDATE ITEMSET**

- ✓ **APRIORI PRINCIPLE**

FREQUENT ITEMSET GENERATION

1



FREQUENT ITEMSET GENERATION

A brute force approach for finding **FREQUENT ITEMSETS** is to determine the **SUPPORT COUNT** for every **CANDIDATE ITEMSET** in the previous structure.

To compute the support count we must compare each candidate itemset against every transaction.

TID	Items
1	bio coke, bread, shrimps
2	peppers, cheddar, salmon, beer
3	tea, apple, banana, phone card, bread
4	coffee, sugar, biscuits, diet coke
5	ham, cheddar, chips

apple
apple, banana
apple, ...
apple, ...
apple, ..., ...
...
banana
banana, bio coke
banana, ...
...
sugar, ..., ..., ...

Ω

apple
banana
...
...
...
salmon
sugar

Set the support count for apple to 1

$\sigma(\text{apple})=1$

CANDIDATE ITEMSETS

FREQUENT ITEMSET GENERATION

2

A brute force approach for finding **FREQUENT ITEMSETS** is to determine the **SUPPORT COUNT** for every **CANDIDATE ITEMSET** in the previous structure.

To compute the support count we must compare each candidate itemset against every transaction.

TID	Items
1	bio coke, bread, shrimps
2	peppers, cheddar, salmon, beer
3	tea, apple, banana phone card, bread
4	coffee, sugar, biscuits, diet coke
5	ham, cheddar, chips

apple
apple, banana
apple, ...
apple, ...
apple, ..., ...
...
banana
banana, bio coke
banana, ...
...
sugar, ..., ..., ...

apple
banana
...
...
...
salmon
sugar

d

Set the support count for apple to 1

$$\sigma(\text{apple})=1$$

Set the support count for apple, banana to 1

$$\sigma(\text{apple, banana})=1$$

Requires $O(NMw)$ comparisons, where:

- N = number of transactions
- w = maximum transaction width

CANDIDATE ITEMSETS

$$M = 2^d - 1$$



Two main approaches to reduce the computational cost of **FREQUENT ITEMSET GENERATION**:

- Reduce the number of **CANDIDATE ITEMSETS (M)**. The **APRIORI PRINCIPLE** is an effective way to eliminate some of the candidate itemsets without counting their support values.
- Reduce the number of comparisons. Instead of matching each candidate itemset against every transaction, we can reduce the number of comparisons by using more advanced data structures, either to store the candidate itemsets or to compress the data sets.

Two main approaches to reduce the computational cost of **FREQUENT ITEMSET GENERATION**:

- Reduce the number of **CANDIDATE ITEMSETS (M)**. The Apriori principle is an effective way to eliminate some of the candidate itemsets without counting their support values.

APRIORI PRINCIPLE. *If an itemset is frequent, then all of its subsets must also be frequent.*

